

Tutorium 3

Typklassen & Algebraische Datentypen

Grundlagen der Praktischen Informatik

15. Juni 2026

Typklassen

Algebraische Datentypen (ADT)

Exkurs: Aussagenlogik

Praxis

Warum Typklassen?

Ohne Typklassen müssen wir für jeden primitiven Datentyp (z. B. Integer, Double) eine eigene Funktion schreiben, auch wenn die Logik identisch ist.

Das Problem

```
1 greaterFive :: Integer -> Bool
2 greaterFive x = x > 5
3
4 greaterFive' :: Double -> Bool
5 greaterFive' x = x > 5
```

Lösung: Wir verallgemeinern die Funktion mit **Typklassen** (vgl. Interfaces in Java).

Verwendung von Typklassen (Num, Eq, Ord)

Die Lösung mit Typklassen

```
1  -- Erlaubt alle numerischen Typen, die ordnerbar sind
2  greaterFive'' :: (Ord a, Num a) => a -> Bool
3  greaterFive'' x = x > 5
4
5  numberToString :: (Eq a, Num a) => a -> String
6  numberToString 0 = "null"
7  numberToString _ = "andere Zahl"
```

- ▶ Num: Numerische Operationen (z. B. Integer, Double, Float).
- ▶ Eq: Erlaubt Überprüfung auf Gleichheit (==).
- ▶ Ord: Erlaubt Vergleiche (>, <). Wichtig: Nicht alles, was Eq ist, ist auch Ord (z. B. Komplexe Zahlen).

Aufzählungsdatentypen (Summen-Typen)

Analog zu `enum` in Java zählen wir eine Menge von exakten Werten auf.

Beispiel: Wochentag

```
1 data Wochentag = Montag | Dienstag | Mittwoch | Donnerstag
2               | Freitag | Samstag | Sonntag
3 deriving (Show, Eq)
4
5 printWochentagZahl :: Wochentag -> Integer
6 printWochentagZahl Montag = 1
7 printWochentagZahl _      = 0
```

Info: `deriving (Show, Eq)` sorgt dafür, dass Haskell die Typen automatisch in der Konsole ausgeben (`Show`) und vergleichen (`Eq`) kann.

Kombinieren verschiedene Werte zu einem neuen komplexen Typ (vgl. Klassen in Java).

Beispiel: Person

```
1 data Person = Person String Integer
2   deriving (Show, Eq)
3
4 printName :: Person -> String
5 printName (Person name alter) = name
```

Rekursive Datentypen (Visualisiert)

Ein ADT kann sich selbst referenzieren, um komplexe Strukturen aufzubauen, z. B. einen Baum.

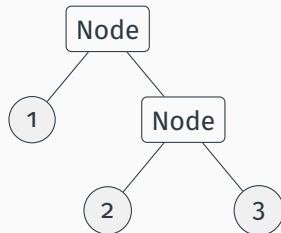
Beispiel: Baum (Tree)

```
1 data Tree = Leaf Integer
2           | Node Tree Tree
```

Instanz:

```
Node (Leaf 1)
  (Node (Leaf 2) (Leaf 3))
```

Speicherdarstellung (Abstrakt)



Vordefinierte Typen: Maybe und Either

Maybe a

```
1 data Maybe a = Nothing | Just a
```

Verhindert Exceptions, indem ein Wert `Just x` oder eben `Nothing` zurückgegeben wird.

Either a b

```
1 data Either a b = Left a |  
    Right b
```

Kann für zwei verschiedene Rückgabetypen verwendet werden (oft: `Left` = Error, `Right` = Resultat).

- ▶ Logische Ausdrücke lassen sich in **Wertetabellen** auswerten.
- ▶ Bei 3 Eingabevariablen (a , b , c) hat die Tabelle exakt $2^3 = 8$ Zeilen.
- ▶ **Äquivalenz:** $A \Leftrightarrow B$ ist genau dann wahr, wenn beide Wahrheitswerte übereinstimmen:

$a \Leftrightarrow b$ ist äquivalent zu $(a \ \&\& \ b) \ || \ (\text{not } a \ \&\& \ \text{not } b)$

Live Demo



`typklassen.hs` & `algebraischeDatentypen.hs`

Wir programmieren Typklassen und eigene ADTs zusammen im Code!

Aufgaben:

1. `numberToMaybeString` (Fehlervermeidung durch `Maybe`)
2. Gerade Zahlen in `Left`, ungerade in `Right` packen (`Either`)
3. Logikrätsel gemeinsam durchgehen